Technical Explanation Report

Distributed and Streaming-based Client-Server Collaboration Application

Evidence: 2026-10-31T07:37:37+00:00 | 273 tests | OK

1. Objective and system architecture

Collab LAN provides persistent username/password accounts, online discovery, private/broadcast chat, and reliable binary file transfer. Clients use raw TCP/UDP sockets; browsers use a local standard-library HTTP bridge. Network, application state, and user interface responsibilities remain separate.

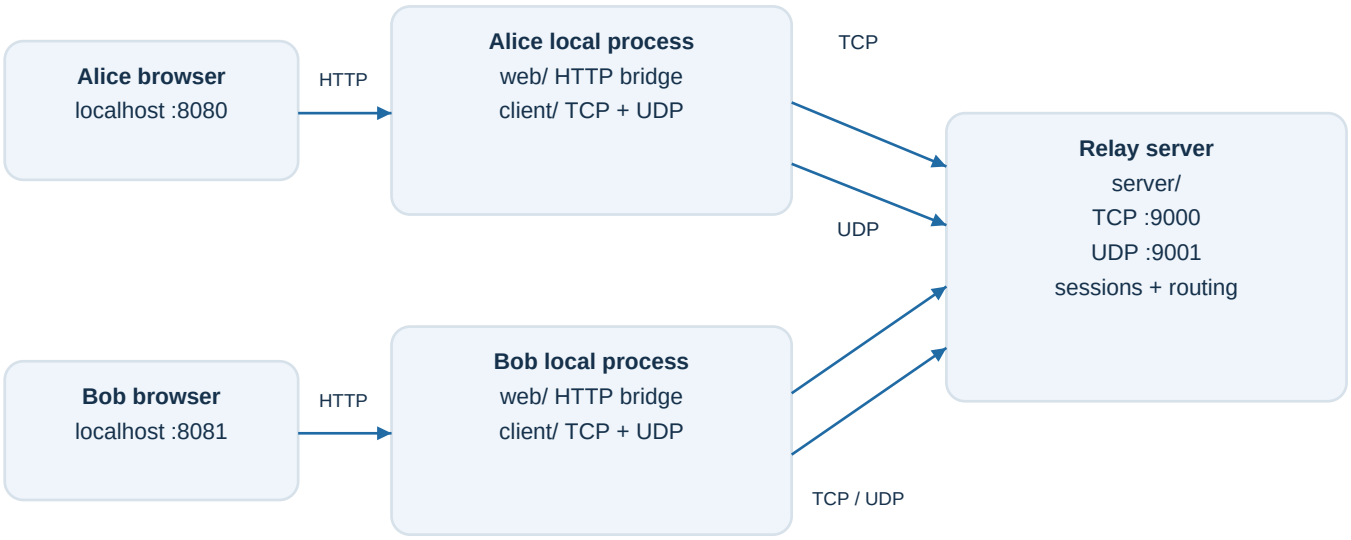

Figure 1. Each browser communicates with its own loopback HTTP backend. Client processes connect to the relay using verified TLS/TCP control and encrypted UDP data. The server decrypts source content and encrypts it for the destination; this is not end-to-end encryption.

Transport interpretation

The assignment's goal and file-transfer rubric require UDP streaming and reassembly; its technology overview also mentions TCP reliable transfer. This project uses TCP for TLS authentication/control/results and UDP for application payloads, discovery, and ACK/retry. Presence broadcast is server fan-out, not LAN broadcast autodiscovery. These interpretations remain instructor questions; no instructor approval is claimed.

2. Architecture, concurrency, and resource ownership

Module	Responsibility
client/	Session lifecycle, terminal menu/commands, bounded chat ordering, outgoing confirmation and file/disk workers.
server/	Account admission, session registry, per-session TCP writers, relay queues and recipient-confirmed file outcomes.
transport/	TLS contexts, account handshake, TCP frames, encrypted UDP envelopes and path-local CUBIC controllers.
shared/	Schemas, limits, bounded worker pools, retry/backoff and independent cleanup.
accounts.py / certgen.py	Parameterized SQLite account operations, Argon2id hashes, bounded authentication policy, and a local CA/server certificate generator.
web/	Loopback HTTP requests call the same client core; JavaScript renders online users, messages and per-transfer progress.

Workers and locks

Server workers accept TCP, receive UDP, schedule relay retries, expire sessions, and publish presence. An event-driven failure monitor owns fatal-worker cleanup without polling for lifecycle ownership; each session has a TCP handler and writer. Clients own TCP receive, UDP receive, heartbeat, and receive-disk workers. HTTP has two send workers/four waiting slots, two upload slots, and eight handlers/sixteen admitted connections.

Registry locks protect short state changes. Session gates are acquired in token order; socket sends and joins occur outside the server registry lock. CUBIC state has its own lock and never sleeps or calls back into application locks. The receive worker owns temporary streams; socket/file cleanup retains unfinished resources for a later close within a shared shutdown deadline. HTTP workers use a single FIFO queue; pending jobs are cancelled at stop and their cleanup is attempted once, with failures reported rather than retried.

Production configuration and scaling

Defaults remain 64 TCP connections, 8,192 pending packets, and 1,024 pending packets per recipient. Authentication permits ten attempts per source IP per 60 seconds. The 20-user same-IP demonstration explicitly starts the production Server with auth_limit=60 (CLI --auth-limit 60), covering twenty registrations plus twenty logins without a test-subclass override. The default security policy is tested separately and is not silently disabled.

3. Application protocol specification

Message / transport	Fields and purpose
register / registered (TLS TCP)	Request: name, password. Success: type=registered, then connection closes. Registration returns to the terminal menu; it does not create an online session.
login / welcome (TLS TCP)	Request: name, password. Welcome: token, session, key, udp_port. Keys and identifiers are freshly random; credentials are not command-line arguments.
error / logout / users (TCP)	Bounded error message; logout ends the session. Users carries names[] and revision. Established controls also use an ordered session envelope inside TLS.
chat_status / file controls (TCP)	Final recipients, delivered names and failure reasons. file_receipt confirms successful save/hash verification; abort/cancel retire outstanding work.
heartbeat / presence (UDP)	Client from+token; server names[]+revision. Verified application activity refreshes liveness; older presence revisions are ignored.
path_challenge / path_response (UDP)	A fresh nonce and initial endpoint must match before application data is accepted. The verified address stays fixed until a new login. Path checks alone do not extend heartbeat life.
data/chat (UDP)	id, from, to, kind, payload, meta. Relay metadata includes sender_session, original channel and stream sequence.
file_start / chunk / end (UDP)	Start: transfer, filename, size, count, sha256. Chunk: transfer/index/base64 bytes. End: transfer and empty payload. Forwarded packets include sender_session.
ack (UDP)	Hop-local application ID; client ACKs include from+token. New ACKs release congestion flight and update the sending hop's controller.

Framing and size checks

JSON is compact UTF-8. TCP uses a four-byte big-endian length; authentication JSON is protected by TLS and capped at 2,048 bytes. Established JSON is capped at 2,000,000 bytes before envelope encoding. UDP includes a 32-byte public context lookup prefix plus a Fernet token and is limited to 1100 bytes. File chunks are 320 raw bytes. Chat and file preflight now use the same session-framed size calculation as the relay, preventing the previously accepted oversized boundary case.

4. Account security and authenticated envelopes

Clients load an explicit CA and verify the server DNS/IP SAN before sending credentials. TLS 1.2 or later protects the entire TCP connection; there is no insecure bypass. SQLite stores id, unique case-sensitive username, Argon2id password_hash, and created_at. Policy: 8-128 password characters, random library-managed salts, 19 MiB memory, two iterations and parallelism one. Four hashing jobs and bounded login frames/deadlines limit unauthenticated work.

A valid password is checked before the server reports an already-online account. Username reservation is atomic. Each login receives a random Fernet key, public session context and separate authorization token. The key is not derived from the password/hash and is never stored in SQLite or printed. Legacy shared-key helpers and compatibility entry points have been removed; authentication uses only the account/TLS flow.

```
auth TCP = TLS(uint32_be(length) || JSON)
control TCP = TLS(uint32_be(length) || Fernet(header || JSON))
UDP = public_context_hex[32] || Fernet(header || JSON)
header = !B16sBBQ
```

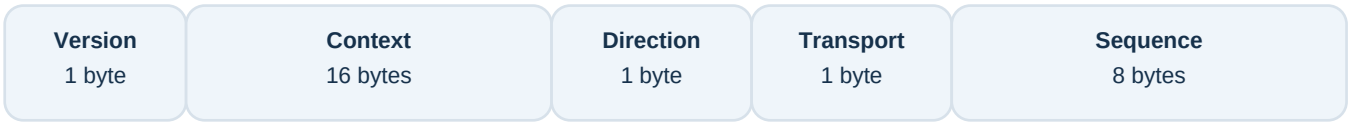

Figure 2. The 27-byte encrypted header binds version 1, 16-byte session context, direction, TCP/UDP transport, and sequence. The outer UDP prefix is only a cipher lookup hint; the inner context must still authenticate.

Replay, endpoint and cleanup boundaries

TCP requires the next sequence. UDP uses a 4,096-packet replay bitmap. Every actual retry retains its application ID but uses a fresh security sequence, allowing legitimate deduplication without accepting replayed frames. UDP schemas and session credentials are checked before endpoint/liveness updates. A challenge response proves initial endpoint reachability; changing the verified UDP address requires a new login. Authentication uses fixed per-IP windows; expired entries are pruned on the next attempt without an LRU cache. Logout, TCP loss, heartbeat expiry, and interrupts retire session references; Python cannot promise physical key-memory erasure. TLS plus a trusted CA prevents an old welcome from authenticating a new TLS connection.

5. Authentication and data lifecycles

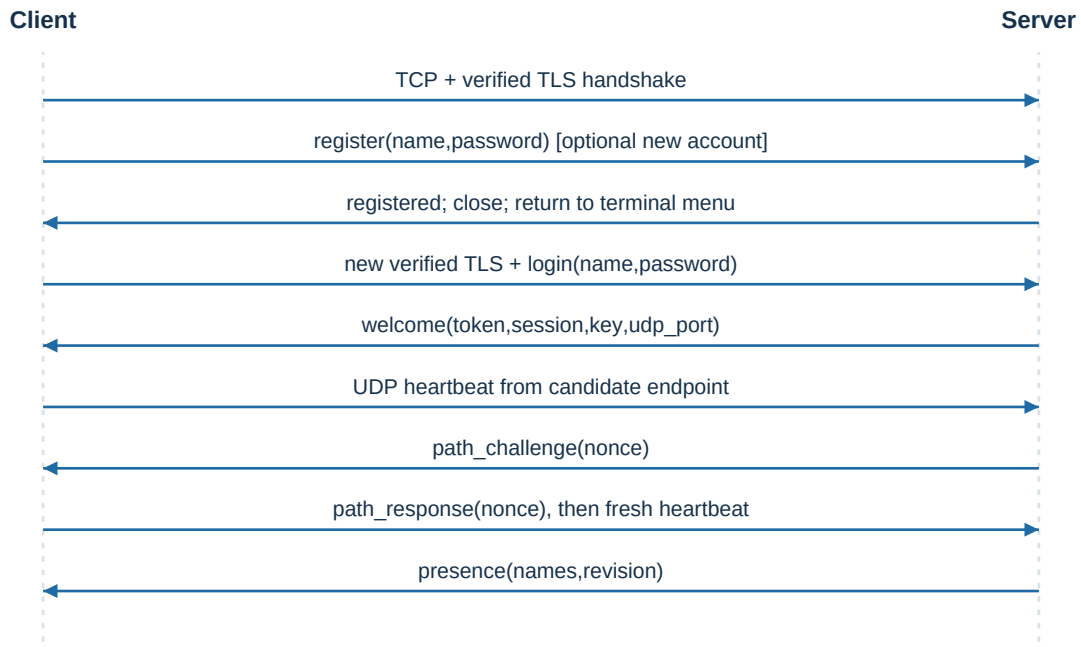

Figure 3. Registration and login are separate TLS connections. Passwords are entered only in the terminal. The Web HTTP listener opens only after successful login. Client connected status follows local UDP setup and worker startup; it does not itself prove path reachability.

Chat and file sequence

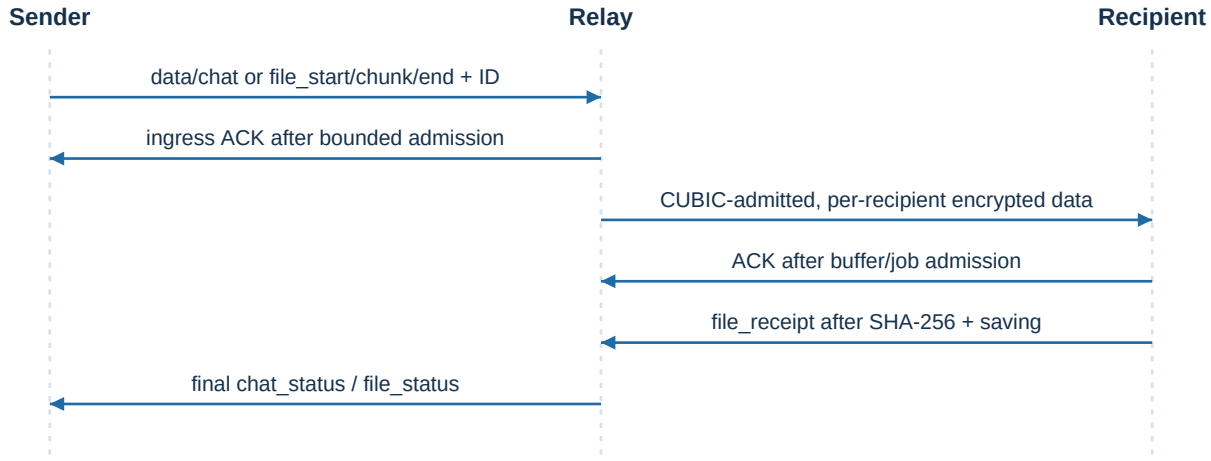

Figure 4. Each UDP data hop has independent congestion state. Hop ACKs indicate admission, not a human read or saved file. Final file success requires verification/save receipts from the original recipients. Sender and recipient sessions are pinned so a replacement login cannot inherit old work.

6. Explicit state-transition diagrams

Client lifecycle

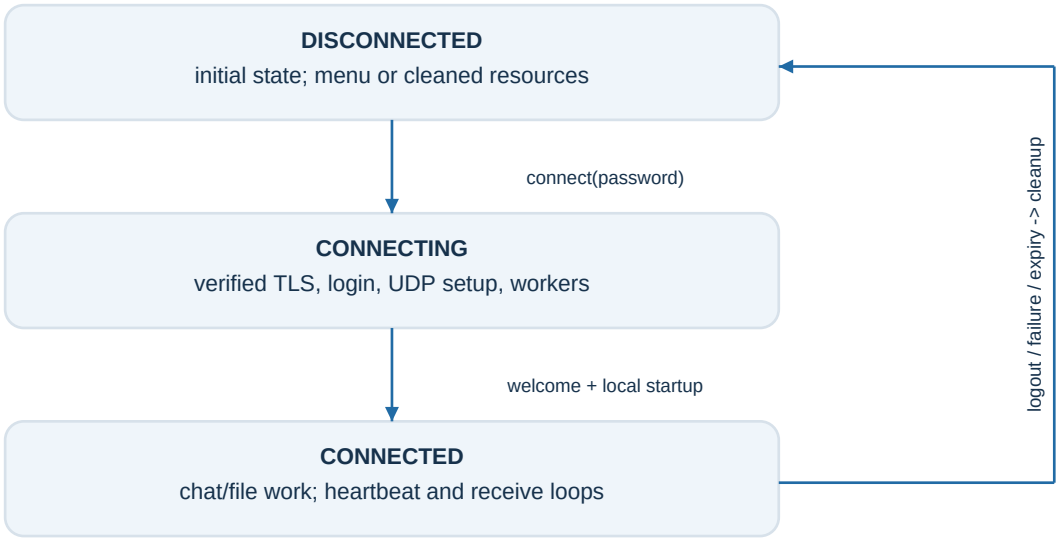

Figure 5. Actual Client.status values are disconnected, connecting and connected. Startup failure also returns from connecting to disconnected. TLS/authentication are internal connecting stages; cleanup has no separately published closing status.

Server lifecycle

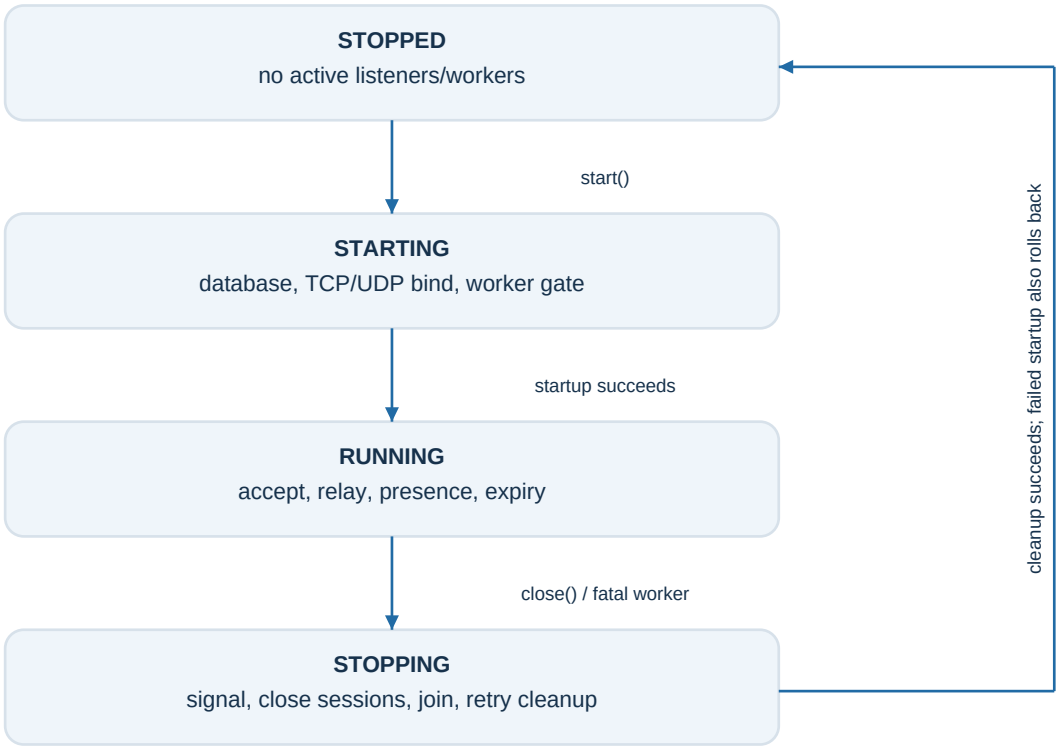

Figure 6. Incomplete cleanup retains stopping status and blocks unsafe restart. These diagrams document the current implementation; they do not claim historical creation before the original code.

7. File transaction states and CUBIC

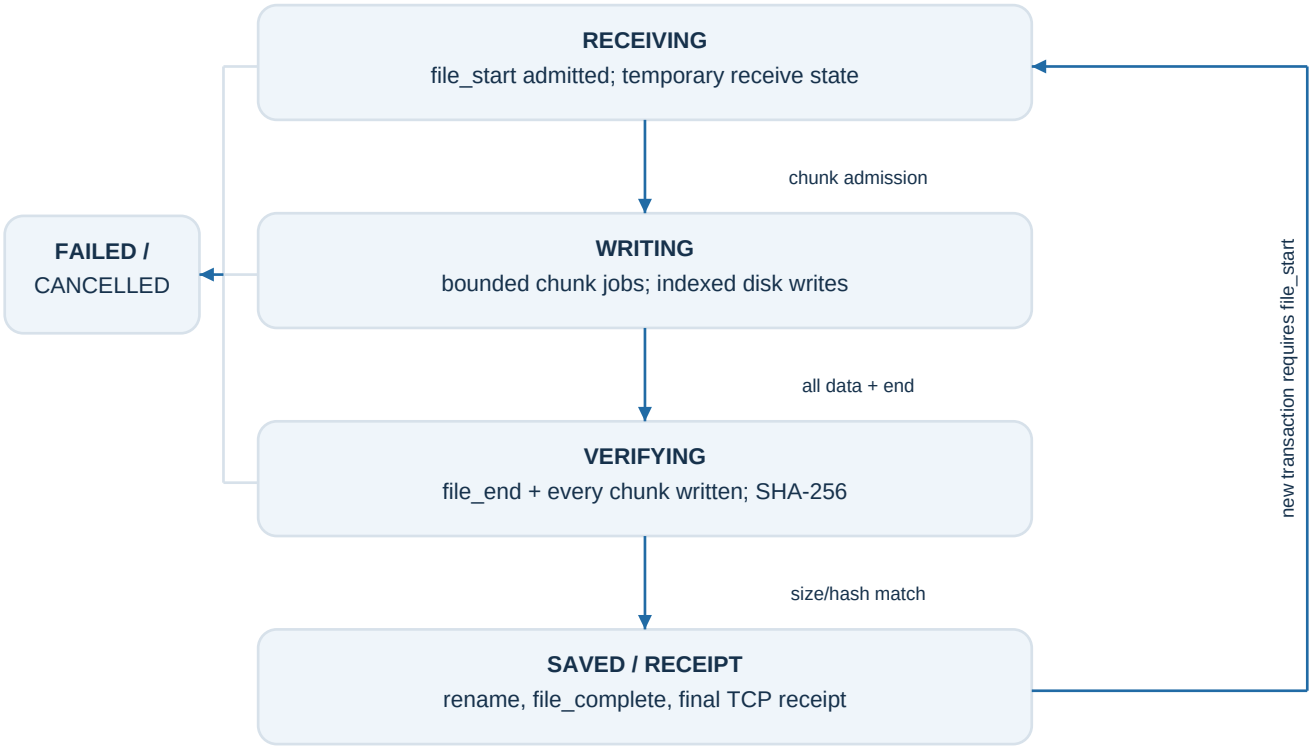

Figure 7. Conceptual receive states are represented by existing ended/finalizing/cancelled flags and job records, not new enum values. Unknown transfers are not ACKed. Duplicate chunks are fingerprint-checked. An active-session failure queues a failed receipt; session cancellation cleans up owned work.

Application-level CUBIC on both UDP hops

Each client session and each server destination session owns a bounded CUBIC controller shared by concurrent application data sends. It uses C=0.4 and beta=0.7, Reno slow start, a cubic epoch with Reno-friendly growth, and fast convergence. Three later ACKs mark a missing flight for fast retransmission; timeout restarts cwnd at one and increases the bounded RTO. First-send RTT samples update smoothed RTT/variation; retransmissions do not supply RTT samples (Karn's rule).

A token bucket limits the average send budget using cwnd/RTT with at most one window per burst. Heartbeats, ACKs and path controls bypass the data window so congestion cannot deadlock control traffic. Client application batches still cap each call at six packets; the independent server relay window can grow up to 128. This is an application-datagram adaptation of CUBIC, not a claim of complete TCP/QUIC conformance: ECN, HyStart++, TCP SACK recovery and spurious-loss undo are absent.

Algorithm reference: RFC 9438, sections 4.2-4.10. https://www.rfc-editor.org/rfc/rfc9438.html. Source: collab/transport/congestion.py.

8. Challenges and solutions

Confidential credentials without losing raw sockets

The former group-key nickname exchange could not establish individual identity. TLS now authenticates the server before credential transfer; SQLite and Argon2id preserve accounts without retaining plaintext passwords. Per-session keys isolate relay envelopes while retaining existing replay/path checks.

Reliable transfer without blocking unrelated streams

One persistent TCP reader preserves fragmented frames across timeouts. UDP data admission and receive-disk work are separate; indexed temporary-file writes handle reordered chunks. Per-chat-stream gap handling bounds waiting without freezing other conversations. Slow-disk and dropped-ACK regression tests check those boundaries, rather than claiming all head-of-line delay disappears.

Real congestion feedback rather than a fixed retry loop

CUBIC state tracks flight, ACKs, loss, RTT, RTO and pacing for both data hops. Window reservation precedes sends; rejected admission consumes neither a retry attempt nor another queue slot. A persistent 3:1 scheduling phase lets files progress even when a small shared destination window limits a batch.

Size accounting must include the actual wire format

The old preflight measured an unprefixed token and could accept a 511-character chat whose relay frame was 1,112 bytes. Preflight now includes the public session prefix and forwarded metadata, matching the 1,100-byte wire ceiling before admission.

A scalable demo must declare its authentication budget

Twenty users on one source IP require forty registration/login requests. A documented --auth-limit 60 setting enables that demonstration; default ten-per-minute behavior remains separately verified. Production scale tests use the real Client and Server classes, not relaxed fixture subclasses.

Cleanup is independent of another cleanup failure

Session identities pin outstanding work; cancellation releases congestion flight. Supervisors signal before joining. Socket shutdown, worker pools, file cleanup and joins share bounded deadlines. Failed sockets/files can be retried on a later close; worker job cleanup runs once and reports failure. No automatic reconnect silently recreates credentials or keys.

9. Test evidence and verification boundaries

Recorded run: **273 tests** in 64.627 seconds; **OK**. Environment: Darwin / Python 3.11.4. Recorded UTC: 2026-10-31T07:37:37+00:00.

```
python -m unittest discover -s tests -v
```

Test-run source snapshot SHA-256: 30fea97cbe0b871cb68f220cf4931d099975548a5dd048a89cf6961e6d8bb1dc

Test module	Count	Evidence
authentication	37	Persistence, password policy, races, TLS trust, fixed rate windows, framing and terminal flow.
chat	31	Broadcast/private results, ordering, missing ACKs and concurrent files.
concurrency	19	Bounded pools, 20 clients, scheduler fairness, slow workers and cleanup.
congestion	13	Deterministic window/RTT/loss/idle behavior and framing regression.
files	35	Chunks, hashes, receipts, reordering, loss, disk failure and streamed source limits.
integration	10	Real encrypted TCP/UDP private/broadcast and binary/empty files.
protocol	12	Wire schemas, invalid payloads, Unicode and boundary sizes.
refactoring	11	Entry points, CLI flags, certificate generation and source grammar.
resilience	36	Killed processes, worker exceptions, deadlines and cleanup retries.
security	31	Session/channel binding, replay windows, path verification and stale packets.
sessions	25	Presence, expiry, disconnect/relogin and endpoint lifecycle.
web	13	HTTP/static routes, progress, JavaScript rendering, bounded admission and cancellation.

Localhost regression tests use OS-assigned ports, temporary databases/certificates/files, fake clocks and controlled loss/error injection. They do not establish WAN fairness or throughput. Historical terminal/LAN-interface checks ran on the same Mac; their supporting evidence files are no longer distributed. A local interface test is not evidence of a second physical device. Windows/Linux execution and instructor approval remain external verification tasks.

10. Reproduction, limits, and open interpretation questions

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m collab.certgen --out certs --names localhost 127.0.0.1
# Server (one terminal)
python -m collab.server --cert certs/server.crt --key certs/server.key
# Alice and Bob (separate terminals; register, then log in)
python -m collab.web --name alice --ca certs/ca.crt --web-port 8080
python -m collab.web --name bob --ca certs/ca.crt --web-port 8081
```

For a 20-user same-IP demonstration, add --auth-limit 60 to the server. For LAN, include the actual server IP in certificate SANs, bind --host 0.0.0.0, securely distribute only ca.crt, and set each client's --host to that IP. HTTP stays on client loopback. The README contains full download-directory and CLI examples.

Boundary	Current limit
Files	25 MiB/file; two incoming and two outgoing; 50 MiB declared incoming total; receive job queue 256.
Congestion	CUBIC flight limit 128 datagrams/path; initial window two; RTO 0.05-2 seconds, plus the application's minimum retry interval.
Application timing	Heartbeat 2 s, expiry 10 s, chat gap wait 1 s, file idle 30 s and total transfer 15 minutes.
Not provided	BBR, offline history, automatic reconnect, resumable files, password reset and browser multi-tab synchronization. BBR is not claimed; CUBIC is the implemented alternative.

Questions requiring instructor confirmation

(1) Does TCP control plus reliable UDP payload delivery satisfy the overview's TCP-transfer wording, given the explicit UDP file rubric? (2) Does server-mediated presence fan-out satisfy discovery broadcast, or is literal LAN broadcast autodiscovery required? (3) Is a documented application-level CUBIC adaptation acceptable, including the disclosed TCP-specific omissions? These are questions, not inferred approvals.

Regeneration and evidence integrity

Install ReportLab as a report-generation dependency if not using a bundled runtime. Save a complete successful unittest run outside the repo, then execute python docs/build_report.py --test-log /path/to/test-results.log. The generator checks source test inventory against the log and refuses stale/mismatched counts. State diagrams are editable in this Python source. Packaging, repository visibility, publication, and submission were not performed.